Práctica 6

Nolan Tannion Rodríguez

Computación Avanzada
Universidad Autónoma de Madrid

Resumen

Se la difusión mediante dos enfoques: resolución numérica de la ecuación de difusión y simulación mediante paseos aleatorios. Se analizan distintas condiciones iniciales y de contorno y casos isótropos y anisótropos. Asimismo, se estudia la evolución de la entropía de Gibbs, observando su crecimiento hasta alcanzar un valor estacionario asociado al equilibrio.

1. Introducción y teoría

1.1. Difusión

La difusión es un proceso fundamental de transporte que describe la evolución temporal de magnitudes conservadas, como la densidad de partículas, la energía o la temperatura, bajo la acción de gradientes espaciales. A nivel macroscópico, este fenómeno se modeliza mediante la ecuación de difusión:

$$\frac{\partial \rho(\mathbf{r},t)}{\partial t} = D\nabla^2 \rho(\mathbf{r},t) \tag{1}$$

donde $\rho(\mathbf{r},t)$ es la densidad, D es el coeficiente de difusión.

Desde un punto de vista microscópico, la difusión puede interpretarse como el resultado colectivo de muchos procesos aleatorios independientes. En particular, un paseo aleatorio conduce, en el límite continuo, a una distribución gaussiana cuya anchura crece con el tiempo. En este contexto, el desplazamiento cuadrático medio cumple:

$$\langle r^2(t)\rangle \propto t \tag{2}$$

lo que permite relacionar directamente el coeficiente de difusión con la dinámica microscópica del sistema.

1.2. Paseos aleatorios

El paseo aleatorio se puede utilizar como modelo para describir procesos estocásticos en física estadística. Consiste en una sucesión de desplazamientos independientes en los que una partícula realiza pasos de longitud fija en direcciones aleatorias. Este modelo permite capturar de forma idealizada la dinámica microscópica de sistemas.

En su versión discreta, el paseo aleatorio se define como la suma de incrementos independientes e idénticamente distribuidos:

$$\mathbf{r}_N = \sum_{i=1}^{N} \Delta \mathbf{r}_i \tag{3}$$

donde cada paso $\Delta\mathbf{r}_i$ es una variable aleatoria. Bajo la hipótesis de independencia y ausencia de sesgo, el valor medio del desplazamiento es nulo,

$$\langle \mathbf{r}_N \rangle = 0 \tag{4}$$

mientras que el desplazamiento cuadrático medio crece linealmente con el número de pasos:

$$\langle r_N^2 \rangle \propto N \tag{5}$$

En dos dimensiones el modelo permite describir fenómenos físicos como el movimiento de partículas suspendidas en un fluido. En este contexto, la relación entre el número de pasos y el tiempo permite establecer la conexión con la ecuación de difusión, identificando el coeficiente de difusión a partir de las propiedades estadísticas del movimiento.

1.3. Difusión y ecuación del calor

Podemos encontrar en [2] la deducción de la ecuación del calor:

$$\frac{\partial T(\mathbf{r},t)}{\partial t} = \alpha \nabla^2 T(\mathbf{r},t) \tag{6}$$

donde $\alpha = \frac{k}{\rho c}$ es la difusividad térmica, con ρ la densidad y c el calor específico del material.

Esta ecuación presenta la misma estructura matemática que una ecuación de difusión.

1.4. Difusión 1D, C. Inhomogéneas

Basándonos en la referencia [1], consideramos la ecuación de difusión unidimensional:

$$\frac{\partial T(x,t)}{\partial t} = D\frac{\partial^2 T(x,t)}{\partial x^2} \tag{7}$$

En el régimen estacionario se cumple $\partial T/\partial t = 0$, por lo que la ecuación se reduce a:

$$\frac{d^2 T(x)}{dx^2} = 0 \tag{8}$$

La solución general de esta ecuación diferencial es

$$T(x) = Ax + B \tag{9}$$

Imponiendo las condiciones de contorno

$$T(0) = T_0 \qquad T(L) = 0 \tag{10}$$

se obtiene

$$B = T_0 \tag{11}$$

$$A = -\frac{T_0}{L} \tag{12}$$

Por tanto, la solución estacionaria viene dada por

$$T(x) = T_0\left(1 - \frac{x}{L}\right) \tag{13}$$

Esta solución describe un perfil lineal de temperatura entre los extremos del sistema, lo que implica un flujo de calor constante en todo el dominio.

1.5. Difusión anisótropa

En muchos sistemas físicos reales, las propiedades del medio dependen de la dirección espacial. Basándonos en [3], este comportamiento se describe mediante la difusión anisótropa:

$$\frac{\partial \phi}{\partial t} = \nabla \cdot (\mathbf{D} \cdot \nabla \phi) \tag{14}$$

Donde $\mathbf{D}$ es el tensor de difusión. En coordenadas cartesianas y con $\mathbf{D}$ diagonal, la ecuación se simplifica a:

$$\frac{\partial \phi}{\partial t} = D_x \frac{\partial^2 \phi}{\partial x^2} + D_y \frac{\partial^2 \phi}{\partial y^2} + D_z \frac{\partial^2 \phi}{\partial z^2} \tag{15}$$

Este tipo de difusión aparece en materiales heterogéneos, cristales anisótropos o medios donde existen direcciones privilegiadas de transporte. En particular, las diferencias entre los coeficientes D_x, D_y y D_z generan una propagación no uniforme de la perturbación inicial, dando lugar a perfiles de difusión elípticos en lugar de esféricos.

La anisotropía puede tener un origen microscópico, relacionado con la estructura del material, o macroscópico, como sistemas sometidos a campos externos.

1.6. Entropía

La entropía es una magnitud fundamental en física estadística que cuantifica el número de configuraciones microscópicas compatibles con un estado macroscópico.

Cuando los microestados no son equiprobables, la entropía viene dada por:

$$S = -\sum_i p_i \ln p_i \tag{16}$$

donde p_i es la probabilidad de cada microestado. Para obtener S con unidades adecuadas se debería multiplicar por k_B aunque en el caso computacional no aporta informamción adicional.

Una propiedad fundamental de la entropía es que en sistemas aislados nunca disminuye, lo que constituye el segundo principio de la termodinámica. Este principio establece la irreversibilidad de los procesos naturales y la tendencia de los sistemas a evolucionar hacia estados de equilibrio, caracterizados por un máximo de entropía.

En el contexto de procesos de difusión, la entropía aumenta con el tiempo debido a la progresiva homogenización del sistema. Por ejemplo, una distribución inicialmente localizada evoluciona hacia una distribución uniforme, incrementando el número de microestados compatibles y, por tanto, la entropía del sistema.

2. Implementación

En ocasiones se utiliza `np.random` para generar números aleatorios por la mayor versatilidad de creación de arrays de números aleatorios.

2.1. Biblioteca de Funciones

Por claridad en los códigos se implementa una biblioteca con varias funciones.

2.1.1. Función `laplace2D`

Resuelve la ecuación utilizando el método de diferencias finitas:

$$\frac{\partial T}{\partial t} = D\left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2}\right) \tag{17}$$

El método utilizado es un esquema explícito en el tiempo (tipo Euler hacia adelante) y centrado en el espacio. La discretización espacial del operador laplaciano se aproxima mediante:

$$\nabla^2 T \approx \frac{T_{i+1,j} + T_{i-1,j} + T_{i,j+1} + T_{i,j-1} - 4T_{i,j}}{dx^2} \tag{18}$$

(asumiendo $dx = dy$ para simplificar la expresión).

El algoritmo actualiza la temperatura en cada punto interior de la malla según:

$$T_{i,j}^{n+1} = T_{i,j}^n + \frac{D\,dt}{dx^2}\left(T_{i+1,j}^n + T_{i-1,j}^n + T_{i,j+1}^n + T_{i,j-1}^n - 4T_{i,j}^n\right) \tag{19}$$

Se implementa la suma mediante slicing del array T para mejorar la eficiencia del cáculo evitando bucles.

Las condiciones de contorno se mantienen implícitamente fijas, porque el bucle solo actualiza los puntos interiores.

El paso temporal dt se escoge, si no se proporciona, según un criterio de estabilidad del esquema explícito en 2D:

$$dt \leq \frac{1}{2D\left(\frac{1}{dx^2} + \frac{1}{dy^2}\right)} \tag{20}$$

Finalmente, la función devuelve el vector de tiempos y la evolución completa de la temperatura en la malla.

```python
def laplace2D(T, tf, t0 = 0, dt=None, dx=1.0, dy=1.0, D = 1):
    """
    Resuelve el lalpaciano de la ecuacion de difusion en 2D y calcula el siguiente paso


    Parametros:
    - T: array 2D [x,y] con condiciones iniciales y/o de frontera
    - dt: paso temporal, opcional, sino se calcula por convergencia
    - dx, dy: pasos espaciales
    - D: parametro de la difusion de laplace

    Devuelve:
    - t: array con el tiempo
    - Sol: array de igual dimension a T con las soluciones
    """

    if dt is None:
        dt = 1/ (2*D* (1/dx**2 + 1/dy**2))    # estabilidad en 2D

    NT = int(np.abs(tf-t0)/dt)

    t = np.arange(t0,tf, dt)


    for n in range(NT-1):
        T[1:-1,1:-1,n+1] = T[1:-1,1:-1,n] + (D*dt/dx**2)*(
            T[2:,1:-1,n] + T[:-2,1:-1,n] +
            T[1:-1,2:,n] + T[1:-1,:-2,n] -
            4*T[1:-1,1:-1,n]
        )

    return t, T
```

2.1.2. Función `random_walk_2d`

La función `random_walk_2d` simula el movimiento de nw caminantes aleatorios en dos dimensiones durante n pasos.

En cada paso temporal, a cada caminante se le asigna una dirección aleatoria uniforme en el intervalo $[0, 2\pi)$, generada mediante un ángulo θ. El desplazamiento en cada eje viene dado por:

$$\Delta x = dr\cos\theta \qquad \Delta y = dr\sin\theta \tag{21}$$

Las posiciones se actualizan de forma acumulativa:

$$x_i^{(k)} = x_i^{(k-1)} + \Delta x \qquad y_i^{(k)} = y_i^{(k-1)} + \Delta y \tag{22}$$

El resultado es una trayectoria bidimensional para cada caminante, donde el módulo del paso es constante (dr) y la dirección es aleatoria en cada iteración. Este modelo es un ejemplo clásico de paseo aleatorio isotrópico, cuya evolución estadística está relacionada con la ecuación de difusión en el límite continuo.

```python
def random_walk_2d(nw, n, dr=1.0, seed=None):
    """
    Random walk 2D con paso de modulo dr.
```

```python

    Params:
    - nw (int) : numero de caminantes
    - n (int)   : numero de pasos
    - dr       : longitud del paso
    - seed      : semilla opcional

    Returns:
        x, y, dist  con dimensiones (nw,n+1)
    """
    if seed is not None:
        np.random.seed(seed)

    # Inicializacion (incluye estado inicial y final)
    x = np.zeros((nw, n + 1))
    y = np.zeros((nw, n + 1))

    for i in range(1, n + 1):
        # Angulos aleatorios
        angles = np.deg2rad(np.random.randint(0, 359, size=nw))

        dx =  dr * np.cos(angles)
        dy =  dr * np.sin(angles)

        x[:, i] = x[:, i-1] + dx
        y[:, i] = y[:, i-1] + dy

    return x, y
```

2.1.3. Función laplace2D_anisot

Para la difución con coeficientes dependientes de la dirección se modifica el bucle sobre NT a:

```python
for i in range(NT-1):
    T[1:-1,1:-1,n+1] = T[1:-1,1:-1,n] + (Dx*dt/dx**2) * (T[2:,1:-1,n] + T[:-2,1:-1,n] - 2*T[1:-1,1:-1,n]) + (Dy*dt/dy**2) * (T[1:-1,2:,n] + T[1:-1,:-2,n] - 2*T[1:-1,1:-1,n])
```

2.1.4. Función random_walk_2d_anisot

En cada paso, cada caminante decide aleatoriamente en qué eje moverse. La probabilidad de moverse en la dirección x es p_x, mientras que en la dirección y es $1 - p_x$. Este mecanismo introduce una asimetría en la dinámica.

Una vez elegido el eje, el caminante se desplaza una distancia fija dr en dirección positiva o negativa con igual probabilidad:

$$\Delta x = \pm dr \qquad \Delta y = \pm dr \tag{23}$$

Este modelo genera una difusión anisótropa efectiva: al existir una probabilidad distinta de desplazamiento en cada eje, el desplazamiento cuadrático medio satisface

$$\langle x^2 \rangle \neq \langle y^2 \rangle \tag{24}$$

lo que equivale a tener coeficientes de difusión distintos ($D_x \neq D_y$).

```python
def random_walk_2d_anisot(nw, n, dr=1.0, Dx=1.0, Dy=1.0, seed=None):
    """
    Random walk 2D anisotropo

    - nw: numero de caminantes
    - n: numero de pasos
    - dr: longitud de paso
    - Dx, Dy: coeficientes de difusion (anisotropia)
    """

    if seed is not None:
        np.random.seed(seed)

    x = np.zeros((nw, n+1))
    y = np.zeros((nw, n+1))

    # Probabilidad de moverse en x
    p = 0.09

    for i in range(1, n+1):

        # elegir eje para cada caminante
        r = np.random.rand(nw)
        move_x = r < p
        move_y = r > p

        # direccion aleatoria +-1
        step_x = dr * np.where(np.random.rand(nw) < 0.5, 1, -1)
        step_y = dr * np.where(np.random.rand(nw) < 0.5, 1, -1)

        # actualizar posiciones
        x[:, i] = x[:, i-1] + move_x * step_x
        y[:, i] = y[:, i-1] + move_y * step_y

    return x, y
```

2.2. Simulación de difusión 1D mediante caminantes aleatorios y diferencias finitas

Se genera la solución en cada paso por diferencias finitas y usando caminantes aleatorios. Las condiciones de contorno inhomogéneas son:

$$T(0,t) = 100 \qquad T(L,t) = 0 \tag{25}$$

2.2.1. Caminantes aleatorios

Cada caminante realiza un paso discreto:

$$x \to x \pm 1 \tag{26}$$

Se aplican condiciones de contorno:

- Se eliminan los caminantes que salen por la derecha ($x \geq Nx$).
- Se refleja el movimiento en el borde izquierdo ($x < 0 \Rightarrow x = 1$).
- Se añaden nuevos caminantes en $x = 0$ para mantener nw constante.

La temperatura se define como el histograma de posiciones:

$$T(x) \propto \text{número de caminantes en } x. \tag{27}$$

Se normaliza a 100 y se suaviza mediante un promedio temporal sobre varios pasos para reducir fluctuaciones.

```python
for i in range(1,NT):
    # Movimiento aleatorio +-1
    mov = np.random.choice([-1, 1], size=pos.size)
    pos = pos + mov

    # Salida en el borde derecho
    inside = pos < Nx
    pos = pos[inside]

    # Reflexion en borde izquierdo
    fuera = np.where(pos < 0)
```

```
    pos[fuera] = 1

    # Nuevos caminantes en x=0
    nn = nw - pos.size
    if nn > 0:
        new_walkers = np.zeros(nn, dtype=int)
        pos = np.concatenate([pos, new_walkers
            ])
```

2.2.2. Diferencias finitas

Resolviendo mediante diferencias finitas:

$$T_j^{n+1} = T_j^n + \frac{D\,dt}{dx^2}\left(T_{j+1}^n - 2T_j^n + T_{j-1}^n\right) \tag{28}$$

Las condiciones de contorno se imponen en cada paso.

```
for j in range(1, Nx-1):
    Td[j, i] = (
        Td[j, i-1]
        + D * dt / dx**2 * (
            Td[j+1, i-1]
            - 2*Td[j, i-1]
            + Td[j-1, i-1]
        )
    )

# Condiciones de contorno
Td[0, i] = 100
Td[-1, i] = 0
```

2.2.3. Promedio temporal

Para estabilizar el ruido del modelo de caminantes, se realiza un promedio sobre una conjunto de tamaño `rango`:

$$T(x,t) = \frac{1}{N}\sum_{k=1}^{N} T_{\text{inst}}(x,t_k) \tag{29}$$

```
# Guardamos en array temporal
temp[:, i % rango] = T_inst

# Promedio temporal cuando hay suficientes
    datos
if i >= rango:
    T[:, i] = np.mean(temp, axis=1)

# 4 primeros pasos
else:
    T[:, i] = T_inst
```

2.3. Condiciones iniciales

Para simular una situación mas realista se utiliza una condición inicial de distribución gaussiana para los caminantes y la densidad.

```
# Densidad
X = np.arange(-NX, NX+1)
Y = np.arange(-NY, NY+1)
X, Y = np.meshgrid(X, Y, indexing='ij')
sigma0 = 5

rho[:, :] = np.exp(-(X**2 + Y**2)/(2*sigma0
    **2))
rho[:, :] /= np.sum(rho[:, :])  # normalizar


# Caminantes
xw = np.zeros((nw, N))
yw = np.zeros((nw, N))

xw[:, 0] = np.random.normal(0, sigma0, size=nw
    )
yw[:, 0] = np.random.normal(0, sigma0, size=nw
    )
```

3. Resultados

3.1. Difusión y caminantes aleatorios

Ejecutando el código podemos ver una simulación comparando evolución temporal de los caminantes aleatorios frente a la resolución por diferencias finitas para cada sistema.

3.1.1. Difusión isótropa

Vemos en la Figura (1) fotograma de la simulación en un tiempo intermedio.

Al ser un sistema simétrico el perfil de difusión toma forma aproximadamente circular en ambos casos.

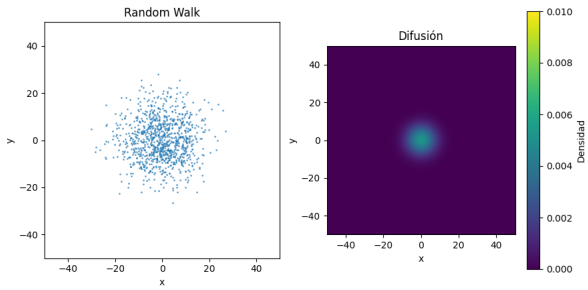

Figura 1: Comparación de la difusión isótropa utilizando diferencias finitas y caminantes aleatorios en un punto intermedio de la simulación.

En la simulación de caminantes aleatorios se ve con más facilidad los outliers que causan un mayor esparcimiento del conjunto mientras que las diferencias finitas dan un perfil más homogéneo de la difusión mostrando que el cúmulo central tiene un valor de campo mucho mayor que los outliers.

El coeficiente de proporcionalidad obtenido para los caminantes aleatorios es:

$$D = 0{,}9898 \pm 0{,}0005$$

La Figura (2) muestra los desplazamientos cuadráticos promedio en cada dirección con sus correspondientes ajustes.

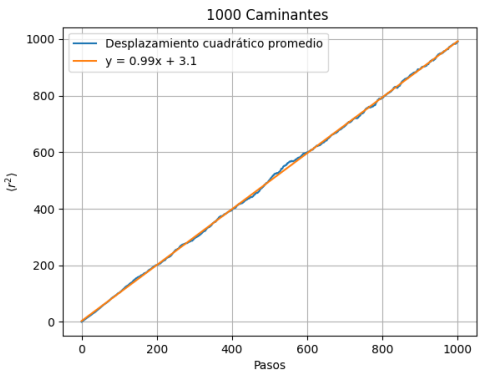

Figura 2: Desplazamientos cuadráticos promedio en el caso isótropo además de las rectas de ajuste para cada uno.

3.1.2. Difusión anisótropa

La Figura (3 representa un fotograma intermedio de la simulación, en este caso para el caso anisótropo.

En este caso no hay simetría y el perfil de difusión toma una forma aproximadamente elíptica en el eje y, donde el coeficiente de difusión es mayor.

Se repite el comportamiento de los outliers más visibles en los caminantes aleatorios y el perfil más homogéneo por diferencias finitas.

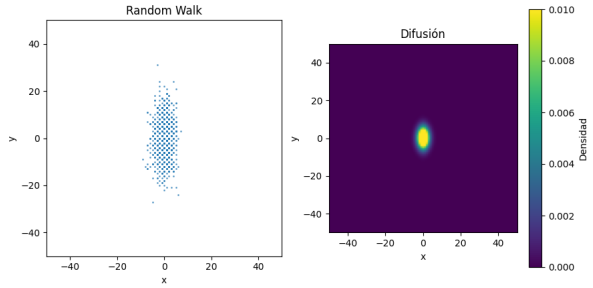

Figura 3: Comparación de la difusión anisótropa utilizando diferencias finitas y caminantes aleatorios en un punto intermedio de la simulación.

Los coeficientes de proporcionalidad obtenidos para los caminantes aleatorios en este caso han sido:

- $D_{r^2} = 0{,}9695$
- $D_{x^2} = 0{,}0923$
- $D_{y^2} = 0{,}8772$

La Figura (4) muestra los desplazamientos cuadráticos promedio en cada dirección con sus correspondientes ajustes.

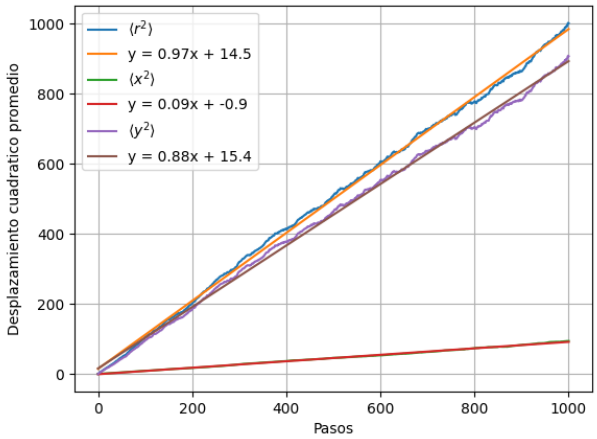

Figura 4: Desplazamientos cuadráticos promedio en el caso anisótropo además de las rectas de ajuste para cada uno.

3.1.3. Gaussiana

De nuevo la Figura (5) representa un fotograma intermedio de la simulación, en este caso para el caso anisótropo.

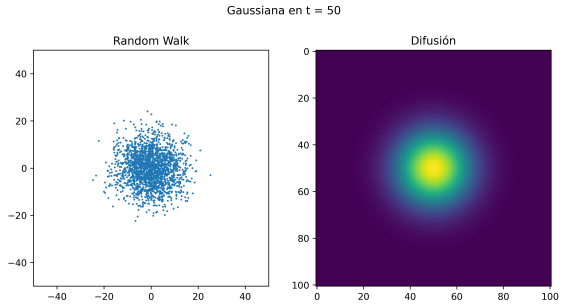

Figura 5: Comparación de la difusión isótropa con condición inicial gaussiana utilizando diferencias finitas y caminantes aleatorios en un punto intermedio de la simulación.

Partiendo de estas condiciones las diferencias finitas y los caminantes aleatorios muestran un perfil de esparcimiento con mayor similitud.

El coeficiente de proporcionalidad obtenido en este caso es:

$$D = 1{,}0018 \pm 0{,}0005$$

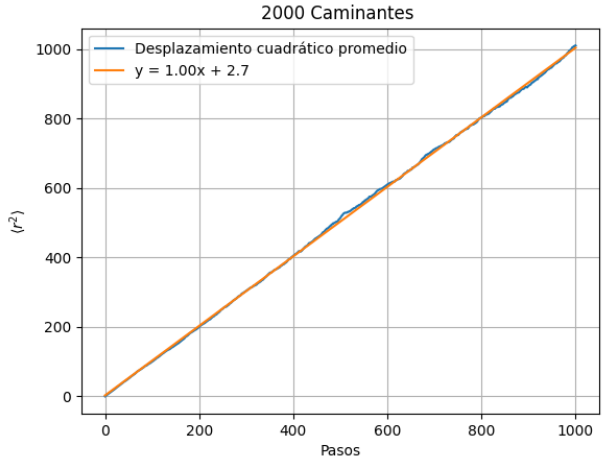

Figura 6: Desplazamientos cuadráticos promedio en el caso isótropo con condición inicial gaussiana, además de las rectas de ajuste para cada uno.

La Figura (6) muestra los desplazamientos cuadráticos promedio en cada dirección con sus correspondientes ajustes.

3.2. Entropía

Las Figuras (7), (8) y (9) representan la entropía de Gibbs para cada paso. Vemos que el paseo aleatorio simula de forma adecuada el comportamiento entrópico del sistema, llegando a una meseta en el máximo de entropía para todos los casos.

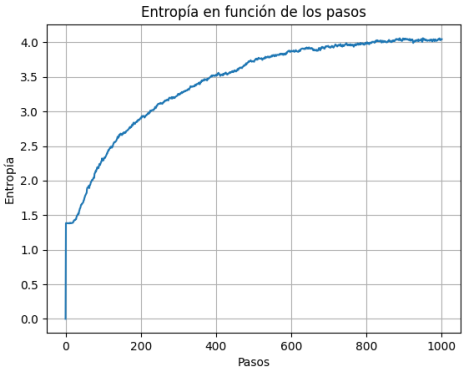

Figura 7: Evolución de la entropía en función del número de pasos para un sistema isótropo.

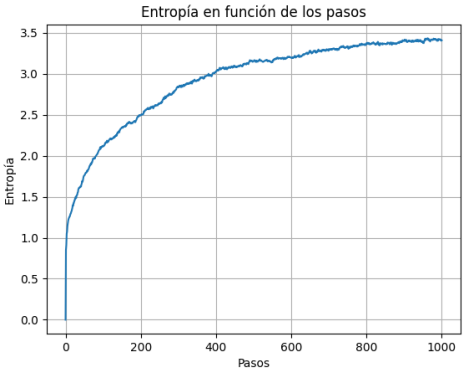

Figura 8: Evolución de la entropía en función del número de pasos para un sistema anisótropo.

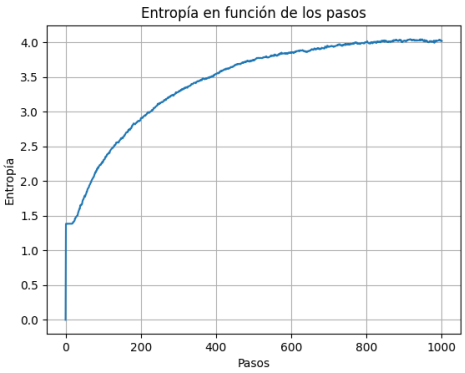

Figura 9: Evolución de la entropía en función del número de pasos para un sistema isótropo con condición inicial gaussiana.

3.3. Difusión inhomogénea

Ejecutando el script se obtiene la evolución temporal por cada uno de los métodos. La Figura (10) muestra el perfil de temperaturas final. Vemos que mediante diferencias finitas obtenemos un perfil coincidente al inicio y al final y se separa en la zona central.

Con los caminantes aleatorios se obtiene un perfil que sigue la tendencia de la solución, sin embargo este tiene muchas oscilaciones. Esto se debe al ser el tamaño del paso comparable con la longitud de la región simulada. Pese a tomar promedios espaciales y temporales las oscilaciones siguen siendo muy notables. Para mejorar el resultado se podría aumentar el número de caminantes para suavizar los promedios o disminuir la distancia del paso.

En este caso no se ha podido conseguir una resolución suficiente en un tiempo de ejecución razonable.

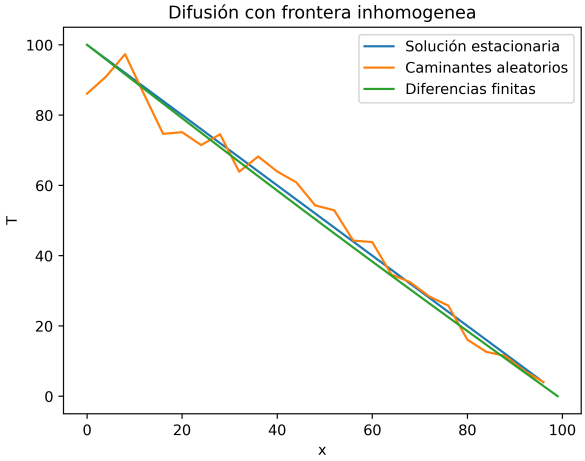

Figura 10: Perfil de temperaturas final tras simular la evolución temporal de los caminantes aleatorios y el calculo por diferencias finitas de condiciones de frontera inhomogéneas.

4. Conclusiones

Se ha podido comprobar que los caminantes aleatorios siguen una ley de difusión, compartiendo el comportamiento tanto en el caso isótropo como el anisótropo. Se obtiene un resultado especialmente similar a la solución por diferencias finitas utilizando una gaussiana, una condición inicial más suave que una delta de Dirac. Finalmente se ha observado la tendencia correcta en el caso de condiciones de frontera inhomogéneas, sin embargo la solución en este caso ha sido menos limpia y más costosa computacionalmente.

Referencias

[1] Farkhad G. Aliev and Antonio Lara. *Problemas resueltos de métodos matemáticos de la física: método de Fourier*. Universidad Autónoma de Madrid, 2019.

[2] Equipo LUA. Ecuación del calor. https://mat.caminos.upm.es/wiki/Ecuaci%C3%B3n_del_calor,_equipo_LUA, 2024. Consultado en marzo de 2026.

[3] Crispin W. Gardiner. *Handbook of Stochastic Methods*. Springer, 4 edition, 2009.